

გაკვეთილი 2 Mod 4 (4.1)

გაკვეთილი 2 — TypeScript-ის ტიპები, პარამეტრები და პრაქტიკული გამოყენება

გაკვეთილის მიზანი

მოსწავლემ უნდა შეძლოს:

- ძირითადი ტიპების გამოყენება;
- მასივის ტიპის განსაზღვრა;
- ფუნქციის პარამეტრების ტიპების მითითება;
- ფუნქციის დაბრუნებული მნიშვნელობის ტიპის განსაზღვრა;
- TypeScript-ის მარტივი კონფიგურაციის გამოყენება;
- TypeScript-ში პატარა პროგრამების დაწერა.

1. წინა გაკვეთილის გამეორება

დასაწყისში ჰკითხე:

რა არის TypeScript?

JavaScript-ის გაფართოებული ვერსია, რომელსაც აქვს ტიპების სისტემა.

რას ნიშნავს ეს?

```
let age: number = 20;
```

აქ `number` გვიჩვენებს, რა ტიპის მონაცემი უნდა იყოს `age`.

გაიმეორეთ:

```
let name: string = "Nika";  
let age: number = 16;  
let isStudent: boolean = true;
```

2. Type inference

TypeScript ყოველთვის არ გვავიძულებს ტიპი ხელით დავწეროთ.

მაგალითად:

```
let name = "Giorgi";
```

TypeScript თვითონ ხვდება, რომ `name` არის `string`.

ამიტომ შემდეგ:

```
name = 100;
```

შეცდომა იქნება.

ასევე:

```
let age = 20;
```

TypeScript ხვდება, რომ `age` არის `number`.

ამას ეწოდება **Type Inference** — ტიპის ავტომატურად განსაზღვრა.

3. მასივები TypeScript-ში

JavaScript-ში უკვე ვიცით:

```
let numbers = [10, 20, 30];
```

TypeScript-ში შეგვიძლია ვთქვათ, რომ მასივში მხოლოდ რიცხვები უნდა იყოს:

```
let numbers: number[] = [10, 20, 30];
```

ტექსტების მასივი:

```
let names: string[] = ["Ana", "Nika", "Giorgi"];
```

Boolean მასივი:

```
let answers: boolean[] = [true, false, true];
```

თუ დავწერთ:

```
let numbers: number[] = [10, 20, "hello"];
```

TypeScript შეცდომას გვაჩვენებს.

4. ფუნქციები TypeScript-ში

JavaScript-ში შეიძლება გვქონდეს:

```
function add(a, b) {  
    return a + b;  
}
```

TypeScript-ში შეგვიძლია ვთქვათ, რომ `a` და `b` რიცხვებია:

```
function add(a: number, b: number) {  
    return a + b;  
}
```

გამოყენება:

```
console.log(add(5, 3));
```

შედეგი:

```
8
```

მაგრამ:

```
add("hello", 3);
```

შეცდომაა.

რატომ?

რადგან ფუნქცია ელოდება:

```
number
```

ტიპის ორ პარამეტრს.

5. ფუნქციის დაბრუნებული მნიშვნელობის ტიპი

შეგვიძლია მივუთითოთ, ფუნქცია რას აბრუნებს.

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

ბოლო:

```
: number
```

ნიშნავს:

ამ ფუნქციამ საბოლოოდ რიცხვი უნდა დააბრუნოს.

მაგალითად ტექსტური ფუნქცია:

```
function greet(name: string): string {  
    return "Hello " + name;  
}
```

გამოყენება:

```
console.log(greet("Nika"));
```

შედეგი:

```
Hello Nika
```

6. ფუნქცია, რომელიც არაფერს აბრუნებს — void

მაგალითად:

```
function showMessage(message: string): void {  
    console.log(message);  
}
```

`void` ნიშნავს, რომ ფუნქცია მნიშვნელობას არ აბრუნებს.

ის უბრალოდ რაღაც მოქმედებას ასრულებს.

```
showMessage("Hello!");
```

7. TypeScript + if

აქ უკვე შეგვიძლია ძველი ცოდნა TypeScript-ს შევუთავსოთ.

```
let age: number = 18;  
  
if (age >= 18) {  
    console.log("Adult");  
} else {  
    console.log("Minor");  
}
```

აქ `if` იგივეა, რაც JavaScript-ში.

TypeScript ახალ `if`-ს არ გვასწავლის — უბრალოდ მონაცემებს ტიპებს ვუმატებთ.

8. TypeScript + loop

იგივე პრინციპია ციკლშიც:

```
let numbers: number[] = [10, 20, 30, 40];  
  
for (let i: number = 0; i < numbers.length; i++) {  
    console.log(numbers[i]);  
}
```

ან:

```
for (let number of numbers) {  
    console.log(number);  
}
```

9. TypeScript-ის პარამეტრების გამართვა — tsconfig

თუ პროექტში ბევრი `.ts` ფაილი გვაქვს, ყოველ ჯერზე ხელით:

```
tsc script.ts
```

წერა მოუხერხებელია.

TypeScript-ს აქვს კონფიგურაციის ფაილი:

```
tsconfig.json
```

მის შესაქმნელად:

```
tsc --init
```

ეს ბრძანება შექმნის:

```
tsconfig.json
```

ეს არის TypeScript პროექტის პარამეტრების ფაილი.

ბავშვებს ამ ეტაპზე არ სჭირდებათ ყველა პარამეტრის ცოდნა.

მთავარი იდეაა:

`tsconfig.json` -ში ვუთითებთ, როგორ უნდა იმუშაოს TypeScript compiler-მა ჩვენს პროექტში.

მაგალითად ერთ-ერთი პარამეტრია:

```
{  
  "compilerOptions": {  
    "strict": true
```

```
}  
}
```

`strict: true` ნიშნავს, რომ TypeScript უფრო მკაცრად შეამოწმებს ჩვენს კოდს.

10. პრაქტიკული მაგალითი

დავწეროთ პატარა პროგრამა:

```
let studentName: string = "Nika";  
let scores: number[] = [80, 90, 70];  
  
function calculateAverage(numbers: number[]): number {  
    let sum: number = 0;  
  
    for (let number of numbers) {  
        sum += number;  
    }  
  
    return sum / numbers.length;  
}  
  
let average: number = calculateAverage(scores);  
  
console.log(studentName);  
console.log("Average:", average);  
  
if (average >= 51) {  
    console.log("Passed");  
} else {  
    console.log("Failed");  
}
```

აქ ერთ პროგრამაში მოსწავლე იყენებს:

- `string`;
- `number`;
- `number[]`;
- ფუნქციას;
- პარამეტრს;
- `return` -ს;

- for ციკლს;
- if/else -ს.

ანუ TypeScript-ს ვუკავშირებთ უკვე ნასწავლ JavaScript-ს.

პრაქტიკული ნაწილი — გაკვეთილი 2

სავარჯიშო 1 — ორი რიცხვის ჯამი

შექმენით ფუნქცია:

```
function sum(a: number, b: number): number {  
    return a + b;  
}
```

გამოიძახეთ სხვადასხვა რიცხვებით.

სავარჯიშო 2 — სრულწლოვანების შემოწმება

შექმენით:

```
function checkAge(age: number): string {  
    if (age >= 18) {  
        return "Adult";  
    } else {  
        return "Minor";  
    }  
}
```

შემდეგ:

```
console.log(checkAge(20));  
console.log(checkAge(15));
```

სავარჯიშო 3 — მასივის ჯამი

მოცემულია:

```
let numbers: number[] = [5, 10, 15, 20];
```

შექმენით ფუნქცია, რომელიც ამ რიცხვების ჯამს დააბრუნებს.

მაგალითი:

```
function getSum(numbers: number[]): number {  
    let sum: number = 0;  
  
    for (let number of numbers) {  
        sum += number;  
    }  
  
    return sum;  
}  
  
console.log(getSum(numbers));
```

შედეგი:

50

სავარჯიშო 4 — მოსწავლის შედეგი

შექმენით ცვლადები:

```
let studentName: string = "Ana";  
let score: number = 75;
```

შემდეგ შექმენით ფუნქცია:

```
function getResult(score: number): string {  
    if (score >= 51) {  
        return "Passed";  
    } else {  
        return "Failed";  
    }  
}
```

ბოლოს გამოიტანეთ:

```
console.log(studentName);  
console.log(getResult(score));
```